

Cyber-Physical ATM Automation: Framework & Security (Full Translation)

[Page 1] Architectural Framework & Security Bypass Strategies

[source: 957] REMOTE ATM CASH WITHDRAWAL AUTOMATION

[source: 966] Cyber-Physical Connection & Security Protocol Bypass

- Digital Input Data Packets
- Encrypted Communication Channel (Cyan)
- Automated Process Flow
- Mechanical Security Lock (Orange)
- Physical Hardware Mechanical Actuator (Cash Dispense)

[Page 2] Core Mission: Removing Barriers for Financial Freedom

To overcome physical banking barriers for individuals with limited mobility (disabled, elderly, or bedridden) via cyber-physical automation. The user completes physical ATM transactions without leaving home, using the hands of a field representative and the system's digital manipulation power.

[Page 3] Concept: Digital Asset Simulation

The bank system assumes the transaction is performed personally by the user at the ATM with their own device.

- Eyes and Hands in the Field
- User Identity and Digital Location
- Seamless ATM Integration

[Page 4] High-Level System Architecture

The central cloud infrastructure serves as a bridge that instantly synchronizes the home user's banking session with the field ATM's physical reality.

- Synchronization and GPS Spoofing
- Request and Authorization (Home -> Cloud)
- QR Image Transmission (Field -> Cloud)

[Page 5] Workflow Phase 1: User and Request

1. **Secure Environment Entry:** User logs into the banking app (e.g., YKB) securely via a remote Android instance (Redroid).

2. **Initiating Transaction:** 'Withdraw Cash via QR' command is triggered.
3. **Waiting Pool:** Transaction data and session state are transferred to the virtual pool for matching and suspended.

[Page 6] Workflow Phase 2: Field and Synchronization

1. **Physical Arrival:** Field operator reaches the specific ATM to perform the transaction.
2. **QR Code Capture:** Operator scans the dynamic QR code on the ATM screen with a special application on their own device.
3. **Instant Transmission:** Captured high-resolution QR image is transmitted to the central server (Virtual Pool) within milliseconds.

[Page 7] Workflow Phase 3: Injection and Withdrawal

- **Image Injection:** QR code from the field is broadcast live to the virtual camera of the remote Android instance using V4L2Loopback and FFmpeg.
- **Location Manipulation:** Instant GPS Spoofing is performed via LSPosed modules for the application's location verification. Coordinates are matched with the ATM.
- **Result:** Bank application sees the QR, verifies the location, and the ATM performs the cash dispense.

[Page 8] End-to-End Flow Map

1. Redroid Login (Home)
2. QR Request Pending (Cloud)
3. ATM QR Capture (Field)
4. Image Sync & GPS Spoofing (Cloud)
5. Camera Injection (Virtual Environment)
6. Cash Dispense (Field)

"Physical distance is reset via millisecond data synchronization and deep operating system manipulation."

[Page 9] Architectural Conflict: Solving the SIGSEGV Crisis

Problem: x86_64 Servers. Banking apps crash with SIGSEGV due to translation errors in security libraries.

Solution: Native ARM64 Infrastructure (Oracle Ampere A1 or AWS Graviton). Eliminating the translation layer completely by running the OS and Docker containers directly on ARM.

[Page 10] Bypassing Firewalls: Emulator and Root Detection

Bank App Detection Mechanisms: Root, Custom ROM, and Emulator.

Defense:

1. **Zygisk:** Blunting detection mechanisms at the kernel level by direct injection into the Android system process (Zygote).
2. **Shamiko:** Advanced module hiding Magisk/Root presence from even the most aggressive banking security libraries.
3. **Hide My Applist:** Isolation shield preventing the app from reading other suspicious tools (e.g., fake GPS tools) on the device.

[Page 11] Engineering Solutions Matrix

Problem	Root Cause	Strategy / Technology
App Crash	x86 Translation Error	Oracle Ampere A1 / Native ARM64
No Camera Access	Lack of physical sensor in emulator	V4L2Loopback + FFmpeg
Location Rejection	Incompatibility with ATM location	Kernel-level manipulation via LSPosed
System Bottleneck	Simultaneous heavy apps	Min. 16GB RAM Allocation

[Page 12] Layered Technology Stack

- **Layer 4 (Automation):** Appium (Flow control and UI interaction)
- **Layer 3 (Manipulation):** V4L2Loopback (Camera), FFmpeg (Injection), LSPosed (Location/Security)
- **Layer 2 (Environment):** Redroid (Docker-based Native ARM64 Image)
- **Layer 1 (Infrastructure):** Oracle Cloud ARM / AWS ARM + Ubuntu 22.04

[Page 13] Scope of Work and Tasks

1. Infrastructure Setup: Stable Redroid environment on ARM server.
2. Bypass Development: Zygisk/Shamiko configuration.
3. Pipeline Construction: V4L2Loopback for external QR image injection.
4. Synchronization: Programmatic GPS change via API/Script.
5. Appium Scripting: Automated login and QR scan navigation scripts.

[Page 14] Critical Success Factor and Expectations

This project requires diving into the depths of the operating system. Mandatory competencies: Zygisk-level bypass strategies, Linux Kernel modules, and virtualization architectures.

"Designed to bring human benefit to life through aggressive cyber-engineering."